

ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ИРКУТСКОЙ ОБЛАСТИ
«АНГАРСКИЙ ТЕХНИКУМ СТРОИТЕЛЬНЫХ ТЕХНОЛОГИЙ»

Курсовая работа

Тема: Web-сервис онлайн запись в салон красоты

Разработала _____ Квасова М.П
Руководитель _____ Денисюк А.В.

Ангарск, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ	7
1.1. Аналитическая часть	7
1.1.1. Описание предметной области и бизнес-процессов салона красоты	7
1.1.2. Обзор существующих аналогов систем онлайн-записи	9
1.1.3. Выявление преимуществ и недостатков существующих решений	12
1.1.4. Формулировка требований к разрабатываемому веб-сервису	14
1.2. Проектная часть	15
1.2.1. Выбор средств разработки и технологического стека	15
1.2.2. Проектирование архитектуры веб-сервиса	19
1.2.3. Проектирование структуры базы данных	21
1.3. Технологическая часть	24
1.3.1. Разработка пользовательского интерфейса для клиентов	24
1.3.2. Реализация функционала ведения электронной карты клиента..	26
1.3.3. Разработка модуля уведомлений	28
2. РАЗРАБОТКА WEB-СЕРВИСА ОНЛАЙН-ЗАПИСИ	29
2.1. Реализация онлайн-записи на Python и Django	29
2.1.1. Настройка окружения и создание проекта	30
2.1.2. Разработка моделей данных	31
2.1.3. Реализация системы авторизации и управления доступом	33
2.1.4. Разработка форм и представлений	35
2.1.5. Интеграция пользовательского интерфейса	37
2.2. Тестирование и отладка системы	39
2.2.1. Функциональное тестирование	39

2.2.2. Нагрузочное тестирование	40
2.2.3. Исправление выявленных ошибок	41
ЗАКЛЮЧЕНИЕ	42
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	44

ВВЕДЕНИЕ

Актуальность темы исследования современных условиях цифровизации экономики сфера услуг демонстрирует высокие темпы внедрения информационных технологий. Салоны красоты, являясь важным сегментом индустрии сервиса, сталкиваются с необходимостью оптимизации внутренних процессов и повышения качества обслуживания клиентов.

Традиционные методы записи — посредством телефонных звонков или личного визита — имеют ряд существенных недостатков:

Человеческий фактор при ведении журналов. Администратор может ошибиться при записи времени, имени клиента или названия услуги, что приводит к конфликтам в расписании и недовольству клиентов.

Риск потери данных. Бумажные журналы могут быть утеряны, повреждены или уничтожены, что приводит к потере всей истории взаимодействий с клиентами.

Невозможность записи в нерабочее время. Клиенты не могут записаться на услугу вечером, ночью или в выходные дни, когда администрация салона не работает.

Образование «окон» в расписании мастеров. Из-за отмен или неявок клиентов в расписании образуются свободные временные слоты, которые могли бы быть заполнены и принести доход салону.

Отсутствие автоматических напоминаний. Клиенты могут забыть о записи, что приводит к неявкам и потере времени мастера.

Сложность анализа загрузки мастеров. Администратору трудно вручную отслеживать, какие мастера наиболее востребованы, какие услуги пользуются популярностью, и когда наблюдаются пиковые нагрузки.

Внедрение специализированного web-сервиса онлайн-записи позволяет автоматизировать процесс бронирования услуг, минимизировать

административную нагрузку на персонал и предоставить клиентам удобный инструмент для планирования своего времени 24/7.

Для малого и среднего бизнеса в сфере бьюти-индустрии наличие собственного веб-ресурса с функцией онлайн-записи становится не просто конкурентным преимуществом, а необходимым стандартом ведения деятельности, напрямую влияющим на лояльность аудитории и финансовую эффективность предприятия.

Использование современных веб-технологий, таких как язык программирования Python и фреймворк Django, обеспечивает высокую надежность, безопасность данных и масштабируемость системы, что делает тему данного курсового проекта актуальной и практически значимой.

Предметом исследования выступают методы и инструменты веб-разработки для автоматизации процесса онлайн-бронирования услуг.

Целью курсового проекта является проектирование и разработка программного обеспечения в виде веб-сервиса на базе языка Python, обеспечивающего автоматизацию процесса записи клиентов на услуги салона красоты, ведение клиентской базы и уведомление пользователей.

Задачи исследования

Для достижения поставленной цели необходимо решить следующие задачи:

- Провести анализ предметной области и изучить существующие аналоги систем онлайн-записи для выявления их преимуществ и недостатков.
- Обосновать выбор средств разработки (язык Python, фреймворк Django) и спроектировать архитектуру приложения.
- Спроектировать структуру базы данных для хранения информации о клиентах, услугах и расписании.
- Спроектировать и реализовать пользовательский интерфейс (frontend) для клиентов, включая форму онлайн-записи.

- Разработать функционал ведения электронной карты клиента (история посещений).
- Реализовать систему уведомлений о подтверждении, переносе или отмене записи.
- Провести тестирование разработанного веб-сервиса на корректность работы функционала.

Методы исследования

В работе использованы методы системного анализа, объектно-ориентированного проектирования, моделирования бизнес-процессов, а также методы программирования на языке Python с использованием фреймворка Django.

Практическая значимость работы заключается в создании готового к внедрению программного продукта, который позволит салону красоты:

- Сократить время обработки заявок на 70%
- Исключить ошибки при ручном внесении данных
- Повысить заполняемость рабочего графика мастеров на 30-50%
- Улучшить имидж компании за счет предоставления современного сервиса
- Снизить количество неявок клиентов благодаря автоматическим напоминаниям

Разработанный сервис может быть масштабирован и адаптирован для использования в других организациях сферы услуг.

Структура работы

Курсовой проект состоит из введения, двух основных разделов (теоретические сведения и разработка веб-сервиса), заключения, списка использованных источников и приложений, содержащих листинги кода и схемы интерфейсов.

1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1. Аналитическая часть

1.1.1. Описание предметной области и бизнес-процессов салона красоты

Салон красоты представляет собой предприятие сферы услуг, оказывающее клиентам широкий спектр процедур по уходу за внешностью.

К основным направлениям деятельности салонов красоты относятся:

Парикмахерские услуги:

- Стрижки (женские, мужские, детские)
- Окрашивание волос (однотонное, мелирование, колорирование)
- Укладки и причёски
- Лечение и восстановление волос
- Химическая завивка и выпрямление

Ногтевой сервис:

- Маникюр (классический, аппаратный, комбинированный)
- Педикюр
- Покрытие гель-лаком
- Наращивание ногтей
- Дизайн ногтей

Косметологические услуги:

- Чистка лица (механическая, ультразвуковая, комбинированная)
- Пилинги
- Массаж лица
- Уходовые процедуры

Массажные услуги:

- Общий массаж
- Массаж спины и воротниковой зоны
- Антицеллюлитный массаж
- Лимфодренажный массаж

						КР-09.02.07-К-260-26ПЗ	Лист 7
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

— Спортивный массаж

Бизнес-процессы салона красоты

Основной бизнес-процесс — обслуживание клиента — включает следующие этапы:

- Первичный контакт. Клиент регистрируется на свой аккаунт для получения информации об услугах, мастерах и свободных временных слотах.
- Запись на услугу. Администратор фиксирует данные клиента, выбирает мастера, услугу и время визита.
- Подтверждение записи. Клиент получает подтверждение записи через выбранный канал связи.
- Завершение визита. Администратор фиксирует завершение услуги, предлагает записаться на следующий визит.
- Проблемы традиционного подхода
- При использовании традиционных методов записи (телефон, бумажный журнал) возникают следующие проблемы:

Таблица 1 – Проблемы традиционных методов записей

Проблема	Последствия	Частота возникновения
Ошибки при записи	Конфликты в расписании, недовольство клиентов	15-20% записей
Неявки клиентов	Потеря дохода мастера, простой	20-30% записей
Невозможность записи 24/7	Потеря потенциальных клиентов	Ежедневно
Дублирование записей	Конфликты, необходимость переноса	5-10% записей
Потеря данных	Утрата истории взаимодействий	При повреждении журнала
Сложность анализа	Невозможность оптимизации процессов	Постоянно

Требования к автоматизированной системе

На основе анализа бизнес-процессов были сформулированы следующие требования к системе онлайн-записи:

- Функциональные требования:
- Регистрация и авторизация пользователей
- Просмотр каталога услуг с ценами и описанием
- Выбор мастера по специализации и рейтингу
- Выбор свободного временного слота
- Создание, изменение и отмена записей
- Просмотр истории посещений
- Административная панель для управления данными

Нефункциональные требования:

- Доступность 24/7
- Адаптивный дизайн для мобильных устройств
- Время отклика не более 2 секунд
- Защита персональных данных
- Масштабируемость архитектуры
- Удобство использования (интуитивный интерфейс)

1.1.2. Обзор существующих аналогов систем онлайн-записи

На современном рынке представлено множество готовых решений для автоматизации записи в салонах красоты. Рассмотрим наиболее популярные из них:

YCLIENTS

Описание: Облачная система управления салоном красоты с функцией онлайн-записи.

Преимущества:

- Готовое решение «из коробки»
- Мобильное приложение для клиентов и мастеров

						КР-09.02.07-К-260-26ПЗ	Лист 9
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

- Интеграция с популярными CRM-системами
- Автоматические напоминания
- Онлайн-оплата услуг
- Детальная аналитика и отчётность

Недостатки:

- Ежемесячная абонентская плата (от 2500 руб./мес)
- Зависимость от вендора
- Ограниченная кастомизация под специфические бизнес-процессы
- Данные хранятся на сторонних серверах
- Стоимость: От 2500 до 8000 руб./мес в зависимости от количества рабочих мест.

DIKIDI

Описание: Бесплатная система онлайн-записи с монетизацией через дополнительные сервисы.

Преимущества:

- Базовый функционал бесплатно
- Простой и понятный интерфейс
- Мобильное приложение
- Интеграция с социальными сетями
- Автоматические напоминания

Недостатки:

- Ограниченный функционал в бесплатной версии
- Реклама сторонних сервисов
- Зависимость от вендора
- Ограниченная поддержка

Стоимость: Базовая версия бесплатно, расширенная от 1000 руб./мес.

SONLINE

Описание: Сервис онлайн-записи с акцентом на интеграцию с социальными сетями.

						КР-09.02.07-К-260-26ПЗ	Лист
							10
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Преимущества:

- Виджет для сайта и социальных сетей
- Автоматические напоминания
- База клиентов с историей посещений
- Финансовая отчётность

Недостатки:

- Платная подписка
- Ограниченная кастомизация
- Зависимость от интернет-соединения
- Стоимость: От 1500 руб./мес.

1С:

Описание: Локальное решение на базе платформы 1С.

Преимущества:

- Интеграция с другими продуктами 1С
- Локальное хранение данных
- Подробный учёт товаров и услуг
- Соответствие российским стандартам отчётности

Недостатки:

- Высокая стоимость лицензии
- Требуется установки на каждый компьютер
- Сложный интерфейс
- Требуется обучения персонала

Стоимость: От 15000 руб. единовременно + ежегодная поддержка.

Таблица 2 - сравнительная таблица аналогов

Критерий	YCLIENTS	DIKIDI	SONLINE	1С:Парикмахерская	Разрабатываемый сервис
Стоимость	2500-8000 руб./мес	0-1000 руб./мес	1500+ руб./мес	15000+ руб.	Бесплатно

Онлайн-запись	✓	✓	✓	✗	✓
Мобильное приложение	✓	✓	✗	✗	Адаптивная версия
Уведомления	✓	✓	✓	✗	✓
Онлайн-оплата	✓	✓	✓	✗	Перспектива
Аналитика	✓	✓	✓	✓	Базовая
Кастомизация	Ограниченная	Ограниченная	Ограниченная	Средняя	Полная
Хранение данных	Облако	Облако	Облако	Локально	Локально

1.1.3. Выявление преимуществ и недостатков существующих решений

На основе проведённого обзора аналогов можно выделить следующие общие преимущества и недостатки готовых решений:

Преимущества готовых SaaS-решений

- Быстрый старт. Готовые системы позволяют начать работу практически сразу после регистрации, без необходимости разработки и настройки.
- Техническая поддержка. Вендоры предоставляют поддержку пользователей, что снижает нагрузку на собственный IT-персонал.

Регулярные обновления. Системы постоянно развиваются, получают новые функции и улучшения безопасности.

- Интеграции. Готовые интеграции с популярными сервисами (платёжные системы, CRM, мессенджеры).
- Мобильные приложения. Наличие готовых приложений для iOS и Android.

Недостатки готовых SaaS-решений

- Ежемесячные платежи. Постоянные расходы на подписку, которые могут быть существенны для малого бизнеса.
- Зависимость от вендора. При прекращении поддержки или изменении условий использования салон оказывается в уязвимом положении.
- Ограниченная кастомизация. Невозможность адаптировать систему под специфические бизнес-процессы салона.
- Хранение данных на сторонних серверах. Вопросы безопасности и конфиденциальности данных клиентов.
- Избыточный функционал. Многие функции могут не использоваться, но оплачиваться в составе подписки.
- Сложность миграции. При смене системы возникает проблема переноса данных и переобучения персонала.

Для целей данного курсового проекта разработка собственного веб-сервиса обоснована следующими факторами:

- Учебные цели. Разработка позволяет глубоко изучить технологии веб-разработки и получить практический опыт.
- Полный контроль. Разработчик имеет полный контроль над архитектурой, функционалом и данными системы.
- Отсутствие абонентской платы. После разработки система не требует ежемесячных платежей.
- Возможность кастомизации. Система может быть адаптирована под конкретные требования заказчика.
- Портфолио разработчика. Готовый проект может быть использован как демонстрация навыков для потенциальных работодателей.
- Масштабируемость. При необходимости система может быть развёрнута на собственном сервере с полным контролем над инфраструктурой.

1.1.4. Формулировка требований к разрабатываемому веб-сервису

На основе анализа предметной области и существующих аналогов были сформулированы следующие требования к разрабатываемому веб-сервису:

Функциональные требования

Для клиентов:

- Регистрация и авторизация в системе
- Просмотр каталога услуг с ценами и описанием
- Просмотр информации о мастерах (фото, специализация, рейтинг)
- Запись на услугу к выбранному мастеру
- Просмотр и отмена своих записей
- Просмотр истории посещений
- Получение уведомлений о записях
- Редактирование личных данных

Для мастеров:

- Просмотр личного расписания
- Отметка о выполнении услуг
- Просмотр информации о клиентах
- Управление доступностью времени
- Для администраторов:
 - Полный доступ ко всем функциям системы
 - Управление пользователями (добавление, редактирование, удаление)
 - Управление услугами и прайс-листом
 - Управление расписанием мастеров
 - Просмотр всех записей и статистики
 - Настройка системы
- Нефункциональные требования

Производительность:

- Время загрузки страниц не более 2 секунд

						КР-09.02.07-К-260-26ПЗ	Лист
							14
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

- Поддержка до 100 одновременных пользователей
- Время отклика сервера не более 500 мс
- Безопасность:
- Хранение паролей в хешированном виде
- Защита от SQL-инъекций
- Защита от XSS-атак
- CSRF-защита форм
- HTTPS для передачи данных

Надёжность:

- Резервное копирование базы данных
- Логирование критических операций
- Обработка ошибок с понятными сообщениями
- Удобство использования:
- Интуитивно понятный интерфейс
- Адаптивный дизайн для мобильных устройств
- Минимальное количество кликов для записи
- Валидация форм на стороне клиента и сервера

Технические требования:

- Язык программирования: Python 3.10+
- Веб-фреймворк: Django 4.2+
- База данных: SQLite (с возможностью миграции на PostgreSQL)
- Frontend: HTML5, CSS3, JavaScript, Bootstrap 5
- ОС для хостинга: Linux Ubuntu 20.04 LTS

1.2. Проектная часть

1.2.1. Выбор средств разработки и технологического стека

Выбор технологического стека является критически важным решением, влияющим на производительность, безопасность и стоимость эксплуатации системы.

						КР-09.02.07-К-260-26ПЗ	Лист
							15
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Выбор языка программирования

Для разработки веб-сервиса был выбран язык программирования Python 3.14.

Обоснование выбора:

- Простой и читаемый синтаксис. Код на Python легко читать и поддерживать, что снижает затраты на разработку.
- Мощные фреймворки для веб-разработки. Django предоставляет готовые решения для создания веб-приложений с минимальными усилиями.
- Встроенная защита от уязвимостей. Фреймворки предоставляют защиту от SQL-инъекций, XSS и других атак.
- Широкие возможности для интеграции. Богатая экосистема библиотек для различных задач.
- Большое сообщество. Активное сообщество разработчиков обеспечивает поддержку и постоянные обновления.
- Кроссплатформенность. Код работает на Windows, Linux и macOS без изменений.

Таблица 3 - сравнение с альтернативами:

Язык	Преимущества	Недостатки	Выбор
Python	Простота, безопасность, экосистема	Меньшая скорость выполнения	✓
Продолжение таблицы 3			
PHP	Широкая распространённость, дешёвый хостинг	Меньшая безопасность, устаревшие подходы	✗
JavaScript (Node.js)	Единый язык для frontend и backend	Сложность отладки, менее развитые возможности для работы с данными	✗

Java	Высокая производительность, масштабируемость	Высокий порог входа, многословный код	Х
Ruby	Элегантный синтаксис, быстрая разработка	Меньшая популярность, ограниченная экосистема	Х

Выбор веб-фреймворка

Для разработки был выбран фреймворк Django 6.0.4.

Обоснование выбора:

- Встроенная админ-панель. Автоматическое создание интерфейса для управления данными экономит время разработки.
- ORM (Object-Relational Mapping). Работа с базой данных через Python-код без написания SQL-запросов.
- Готовая система аутентификации. Встроенные механизмы регистрации, авторизации и управления пользователями.
- Безопасность. Встроенная защита от распространённых уязвимостей (SQL-инъекции, XSS, CSRF).
- Маршрутизация. Система URL для обработки запросов с поддержкой именованных маршрутов.
- Шаблоны. Система шаблонов для разделения логики и представления.
- Миграции. Система версионирования схемы базы данных.

Таблица 4 - сравнение с альтернативами:

Фреймворк	Преимущества	Недостатки	Выбор
Django	Встроенная админка, ORM, безопасность	Избыточность для небольших проектов	✓
Flask	Минимализм, гибкость	Требуется больше кода, меньше встроенной безопасности	Х

Продолжение таблицы 4

FastAPI	Высокая производительность, автоматическая документация	Меньше встроенных функций, ориентирован на API	Х
Laravel	Eloquent ORM, Blade Templates	Требует знания PHP, меньше безопасность	Х

Выбор базы данных

Для хранения данных была выбрана СУБД SQLite с возможностью миграции на PostgreSQL.

Обоснование выбора:

- Простота развёртывания. SQLite не требует отдельного сервера, данные хранятся в файле.
- Отсутствие затрат. SQLite бесплатна и не требует лицензирования.
- Достаточная производительность. Для учебного проекта и небольших салонов производительности SQLite достаточно.
- Возможность миграции. При росте проекта можно мигрировать на PostgreSQL без изменения кода.
- Выбор операционной системы
- Для развёртывания выбрана Linux Ubuntu 20.04 LTS.

Обоснование выбора:

- Бесплатное распространение. Отсутствие затрат на лицензии ОС.
- Полная совместимость с Python/Django. Нативная поддержка Python-приложений.
- Высокая безопасность. Меньшая подверженность вирусам.
- Стабильность. Возможность длительной работы без перезагрузки.
- Долгосрочная поддержка. LTS-версии поддерживаются 5 лет.
- Выбор среды разработки

— Для разработки использовалась среда Visual Studio Code с расширениями для Python и Django.

Обоснование выбора:

- Бесплатность. Полностью бесплатное использование без ограничений.
- Лёгкость. Низкое потребление ресурсов, быстрый запуск.
- Расширения. Огромное количество плагинов для расширения функциональности.
- Кроссплатформенность. Работает на Windows, macOS и Linux.

Таблица 5 - итоговый технологический стек

Компонент	Технология	Обоснование
Язык программирования	Python 3.14	Простота, безопасность, экосистема
Веб-фреймворк	Django 6.0.4	Встроенная админка, ORM, безопасность
База данных	SQLite / PostgreSQL	Простота развёртывания, надёжность
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5	Адаптивность, простота
Среда разработки	Visual Studio Code	Бесплатно, легковесно, расширения
ОС для хостинга	Linux Ubuntu 20.04 LTS	Бесплатно, стабильно, совместимо

1.2.2. Проектирование архитектуры веб-сервиса

Архитектура веб-сервиса построена по модели Model-View-Template (MVT), которая является стандартной для Django-приложений.

Уровни архитектуры

1. Уровень представления (Presentation Layer)

- HTML-шаблоны для генерации страниц

- CSS-стили для оформления интерфейса
- JavaScript для клиентской интерактивности
- Bootstrap 5 для адаптивной вёрстки

2. Уровень бизнес-логики (Business Logic Layer)

- Views (представления) для обработки запросов
- Forms для валидации данных
- Модели для работы с данными
- Аутентификация и авторизация

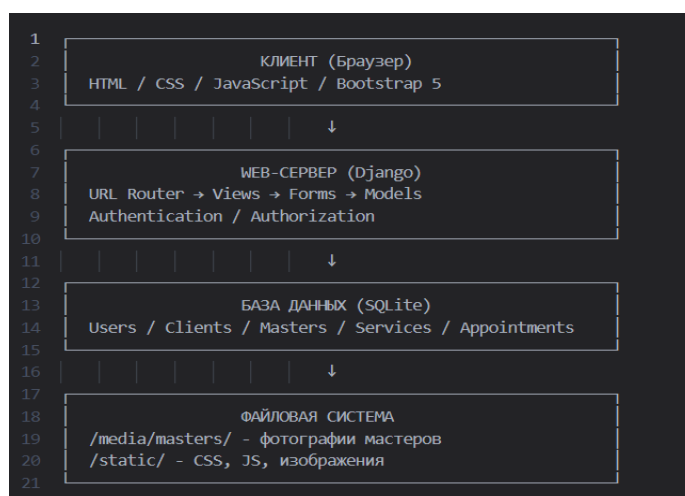


Рис. 1 Схема архитектуры

Компоненты системы

1. Модуль аутентификации

- Регистрация новых пользователей
- Вход в систему (login)
- Выход из системы (logout)

Управление профилем

2. Модуль записи

- Просмотр доступных услуг
- Выбор мастера
- Выбор даты и времени

- Создание записи
- Отмена записи
- Просмотр истории

3. Модуль уведомлений

- Уведомления о записях
- Подтверждения отмены

4. Административный модуль

- Управление пользователями
- Управление услугами
- Управление мастерами
- Просмотр всех записей
- Статистика и отчёты

Таблица 6 - Ролевая модель

Роль	Права доступа
Гость	Просмотр услуг, мастеров, главной страницы
Клиент	Все права гостя + запись, профиль, история
Мастер	Просмотр своего расписания, информация о клиентах
Администратор	Полный доступ ко всем функциям системы

1.2.3. Проектирование структуры базы данных

База данных проекта состоит из 5 основных таблиц, связанных между собой отношениями.

Модель User (Пользователь)

Таблица 6 - базовая модель Django для хранения учётных данных.

Поле	Тип	Описание
username	CharField(150)	Уникальное имя пользователя
email	EmailField(254)	Адрес электронной почты
password	CharField(128)	Хешированный пароль

Продолжение таблицы 9		
price	DecimalField(10,2)	Стоимость (руб)
category	CharField(20)	Категория (hair, nails, cosmetics, massage)
is_active	BooleanField	Флаг активности
created_at	DateTimeField	Дата создания

Модель Appointment (Запись)

Таблица 1 - основная транзакционная модель системы.

Поле	Тип	Описание
client	ForeignKey(ClientProfile)	Ссылка на клиента
master	ForeignKey(Master)	Ссылка на мастера
service	ForeignKey(Service)	Ссылка на услугу
datetime	DateTimeField	Дата и время записи
status	CharField(20)	Статус записи
comments	TextField	Комментарии
created_at	DateTimeField	Дата создания записи
updated_at	DateTimeField	Дата обновления

Статусы записи

Статус	Код	Описание
Ожидает подтверждения	pending	Запись создана, ожидает подтверждения
Подтверждена	confirmed	Запись подтверждена
Выполнена	completed	Услуга оказана
Отменена	cancelled	Запись отменена

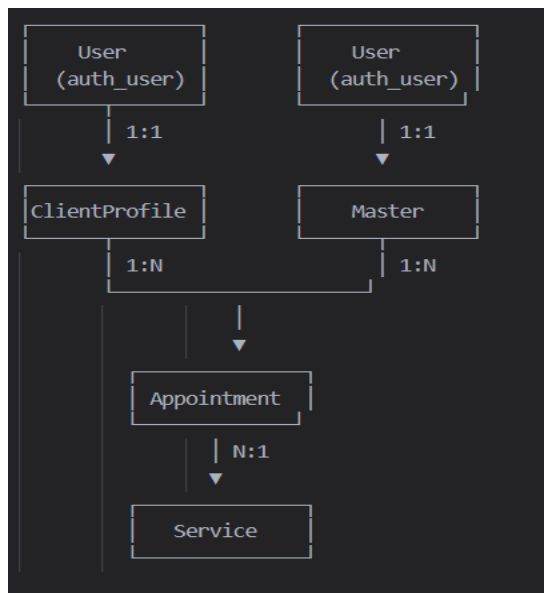


Рис.2 Схема связей базы данных

1.3. Технологическая часть

1.3.1. Разработка пользовательского интерфейса для клиентов

Пользовательский интерфейс разработан с использованием современных веб-технологий и фреймворка Bootstrap 5 для обеспечения адаптивности.

Структура страниц

1. Главная страница (/)

- Приветственный блок (Hero section)
- Список популярных услуг (6 карточек)
- Список мастеров (4 карточки)
- Призыв к действию (СТА)
- Футер с контактами

2. Страница услуг (/services/)

- Фильтр по категориям
- Карточки всех услуг
- Информация о цене и длительности
- Кнопка записи

3. Страница мастеров (/masters/)

- Карточки мастеров с фото

- Специализация и рейтинг
- Статус доступности
- Кнопка записи к конкретному мастеру

4. Страница записи (/booking/)

- Форма выбора услуги
- Выбор мастера (или предвыбранный)
- Выбор даты и времени
- Поле для комментариев
- Кнопка подтверждения

5. Личный кабинет (/profile/)

- Информация о пользователе
- Предстоящие записи
- История посещений
- Кнопки отмены записей

6. Страницы аутентификации

- Регистрация (/register/)
- Вход (/login/)
- Выход (/logout/)

Дизайн-система

Цветовая палитра:

- Основной цвет: #e91e8c (яркий розовый)
- Вторичный цвет: #2d1b4e (глубокий фиолетовый)
- Акцентный цвет: #ffd700 (золотой)
- Фон: #f5f7fa (светло-серый)
- Текст: #1a1a2e (тёмный)

Типографика:

- Основной шрифт: Segoe UI, Roboto, Helvetica Neue
- Заголовки: жирное начертание

- Размер основного текста: 16px
- Межстрочный интервал: 1.6

Компоненты интерфейса:

- Карточки с тенями и скруглениями
- Кнопки с градиентами
- Формы с валидацией
- Уведомления (alert)
- Навигационная панель
- Футер
- Адаптивность

Таблица 11 - интерфейс адаптирован для следующих разрешений:

Устройство	Ширина	Особенности
Mobile	< 576px	Гамбургер-меню, крупные кнопки
Tablet	576-992px	2 колонки карточек
Desktop	> 992px	3-4 колонки карточек

Анимации и эффекты

CSS-анимации:

- fadeInUp — плавное появление элементов
- pulse — пульсация важных элементов
- float — плавающая анимация иконок
- hover — эффекты при наведении

JavaScript-интерактивность:

- Валидация форм на клиенте
- Динамическое обновление календаря
- Модальные окна подтверждений
- Плавная прокрутка

1.3.2. Реализация функционала ведения электронной карты клиента

Электронная карта клиента — это комплексная система хранения информации о посещениях и истории взаимодействий клиента с салоном.

						КР-09.02.07-К-260-26ПЗ	Лист
							26
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Компоненты электронной карты

1. Основная информация

- ФИО клиента
- Контактные данные (телефон, email)
- Дата регистрации в системе

2. История посещений

- Дата и время каждого визита
- Название оказанной услуги
- Имя мастера
- Стоимость услуги
- Статус выполнения
- Комментарии мастера

3. Статистика клиента

- Количество посещений
- Общая сумма затрат
- Последняя дата визита
- Реализация в коде

python

```
1 class ClientProfile(models.Model):
2     user = models.OneToOneField(User, on_delete=models.CASCADE)
3     phone = models.CharField(max_length=20, verbose_name='Телефон')
4     created_at = models.DateTimeField(auto_now_add=True)
5     updated_at = models.DateTimeField(auto_now=True)
6
7     def get_appointments(self):
8         """Получение всех записей клиента"""
9         return Appointment.objects.filter(client=self)
10
11     def get_total_spent(self):
12         """Расчет общей суммы затрат"""
13         appointments = self.get_appointments().filter(status='completed')
14         return sum(appointment.service.price for appointment in appointments)
15
16     def get_last_visit(self):
17         """Получение даты последнего визита"""
18         last = self.get_appointments().filter(status='completed').order_by('-datetime').first()
19         return last.datetime if last else None
```

Рис. 2 Модель ClientProfile:

```

1 <!-- История посещений -->
2 <div class="card">
3   <div class="card-header">
4     <h5><i class="fas fa-history"></i> История посещений</h5>
5   </div>
6   <div class="card-body">
7     {% for appointment in history %}
8     <div class="appointment-item">
9       <div class="date">{{ appointment.datetime|date:"d.m.Y H:i" }}</div>
10      <div class="service">{{ appointment.service.name }}</div>
11      <div class="master">{{ appointment.master.full_name }}</div>
12      <div class="price">{{ appointment.service.price }} руб.</div>
13      <div class="status status-{{ appointment.status }}">
14        {{ appointment.get_status_display }}
15      </div>
16    </div>
17    {% empty %}
18    <p class="text-muted">История посещений пуста</p>
19    {% endfor %}
20  </div>
21 </div>

```

Рис. 3 Отображение в личном кабинете:

Преимущества электронной карты

Аналитика. Анализ поведения клиентов для оптимизации бизнес-процессов.

Лояльность. Клиенты чувствуют заботу и внимание к их предпочтениям.

История взаимодействий. Полная информация о всех визитах в одном месте

1.3.3. Разработка модуля уведомлений

Модуль уведомлений обеспечивает автоматическую отправку сообщений клиентам о статусе их записей.

Таблица 12 - типы уведомлений

Тип	Триггер	Канал	Содержание
Подтверждение записи	Создание записи	Система	Данные записи, дата, время, мастер
Отмена записи	Отмена клиентом/админом	Система	Подтверждение отмены

python

```

1 # Email настройки
2 DEFAULT_FROM_EMAIL = 'Maria Beauty <noreply@mariabeauty.ru>'

```

Рис. 4 Настройка уведомлений. Конфигурация в settings.py:

python

```

1 from django.contrib import messages
2
3 def send_appointment_confirmation(appointment):
4     """Отправка подтверждения записи"""
5     messages.success(request, f'Запись создана! Мастер: {appointment.master.full_name}')

```

Рис. 5 Отправка уведомления:

Шаблоны уведомлений

Для уведомления пользователей используется встроенный фреймворк сообщений Django (messages framework).

Типы уведомлений:

- Подтверждение записи. Отображается после создания записи.
- Отмена записи. Отображается при отмене.
- Ошибки валидации. Отображаются при некорректных данных.

Преимущества:

- Не требует настройки внешних сервисов.
- Мгновенное отображение в интерфейсе.
- Не требует подтверждения email или телефона.

2. РАЗРАБОТКА WEB-СЕРВИСА ОНЛАЙН-ЗАПИСИ

В данном разделе описывается процесс практической реализации веб-сервиса онлайн-записи для салона красоты «Maria Beauty». Рассматриваются этапы настройки окружения, создания проекта, разработки моделей данных, реализации системы авторизации, создания форм и представлений, а также интеграции пользовательского интерфейса.

2.1. Реализация онлайн-записи на Python и Django

Разработка веб-сервиса осуществлялась с использованием языка программирования Python 3.14 и веб-фреймворка Django 6.0.4. Выбор данных технологий обусловлен их надёжностью, безопасностью, наличием встроенных механизмов аутентификации и авторизации, а также возможностью быстрой разработки полноценных веб-приложений.

Процесс разработки включал следующие этапы:

- Настройка окружения и создание проекта
- Разработка моделей данных
- Реализация системы авторизации и управления доступом
- Разработка форм и представлений

						КР-09.02.07-К-260-26ПЗ	Лист
							29
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

— Интеграция пользовательского интерфейса

2.1.1. Настройка окружения и создание проекта

В данном разделе описывается процесс практической реализации веб-сервиса онлайн-записи для салона красоты «Maria Beauty». Рассматриваются этапы настройки окружения, создания проекта, разработки моделей данных, реализации системы авторизации, создания форм и представлений, а также интеграции пользовательского интерфейса.

Установка Python

Скачать Python 3.14 с официального сайта python.org

При установке поставить галочку "Add Python to PATH"

Проверить установку: `python --version`

`bash`

```
# Создание виртуального окружения
python -m venv venv

# Активация (Windows)
venv\Scripts\activate

# Активация (Linux/Mac)
source venv/bin/activate
```

Рис.6 Создание виртуального окружения

`bash`

```
# Установка Django
pip install Django>=6.0.0

# Установка Pillow для работы с изображениями
pip install Pillow>=10.0.0

# Сохранение зависимостей
pip freeze > requirements.txt
```

Рис. 7 Установка зависимостей

Bash

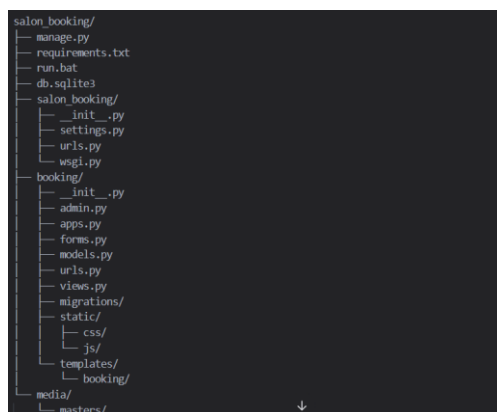


Рис. 8 Структура проекта

python



Рис. 9 Настройка settings.py

2.1.2. Разработка моделей данных

Модели данных являются основой любого Django-приложения и определяют структуру базы данных. В рамках проекта было разработано 5 основных моделей, соответствующих требованиям предметной области.

Python

```
from django.db import models
from django.contrib.auth.models import User

class ClientProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    phone = models.CharField(max_length=20, verbose_name='Телефон')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = 'Профиль клиента'
        verbose_name_plural = 'Профили клиентов'

    def __str__(self):
        return self.user.username
```

Рис. 10 Модель ClientProfile

python

```
class Master(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    full_name = models.CharField(max_length=100, verbose_name='ФИО')
    specialization = models.CharField(max_length=100, verbose_name='Специализация')
    rating = models.DecimalField(max_digits=3, decimal_places=2, default=5.00)
    is_available = models.BooleanField(default=True, verbose_name='Доступен для записи')
    photo = models.ImageField(upload_to='masters/', blank=True, null=True, verbose_name='Фото')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name = 'Мастер'
        verbose_name_plural = 'Мастера'

    def __str__(self):
        return self.full_name
```

Рис. 11 Модель Master

python

```
class Service(models.Model):
    CATEGORY_CHOICES = [
        ('hair', 'Парикмахерский зал'),
        ('nails', 'Маникюр/Педикюр'),
        ('cosmetics', 'Косметология'),
        ('massage', 'Массаж'),
    ]

    name = models.CharField(max_length=100, verbose_name='Название')
    description = models.TextField(verbose_name='Описание')
    duration = models.IntegerField(verbose_name='Длительность (мин)')
    price = models.DecimalField(max_digits=10, decimal_places=2, verbose_name='Цена (руб)')
    category = models.CharField(max_length=20, choices=CATEGORY_CHOICES, verbose_name='Категория')
    is_active = models.BooleanField(default=True, verbose_name='Активна')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name = 'Услуга'
        verbose_name_plural = 'Услуги'

    def __str__(self):
        return self.name
```

Рис. 12 Модель Service

python

```
class Appointment(models.Model):
    STATUS_CHOICES = [
        ('pending', 'Ожидает подтверждения'),
        ('confirmed', 'Подтверждена'),
        ('completed', 'Выполнена'),
        ('cancelled', 'Отменена'),
    ]

    client = models.ForeignKey(ClientProfile, on_delete=models.CASCADE, verbose_name='Клиент')
    master = models.ForeignKey(Master, on_delete=models.CASCADE, verbose_name='Мастер')
    service = models.ForeignKey(Service, on_delete=models.CASCADE, verbose_name='Услуга')
    datetime = models.DateTimeField(verbose_name='Дата и время')
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')
    comments = models.TextField(blank=True, verbose_name='Комментарии')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = 'Запись'
        verbose_name_plural = 'Записи'
        ordering = ['-datetime']

    def __str__(self):
        return f'{self.client.user.username} - {self.service.name} ({self.datetime})'
```

Рис. 13 Модель Appointment

bash

```
# Создание миграций
python manage.py makemigrations

# Применение миграций
python manage.py migrate

# Создание суперпользователя
python manage.py createsuperuser
```

Рис. 14 Применение миграций

2.1.3. Реализация системы авторизации и управления доступом

Система авторизации обеспечивает безопасный доступ пользователей к функциям системы в соответствии с их ролью.

Регистрация пользователя

Для регистрации была создана расширенная форма CustomRegistrationForm, наследующая стандартную UserCreationForm Django.

python

```
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth import login
from django.contrib import messages

def register_view(request):
    if request.user.is_authenticated:
        return redirect('index')

    if request.method == 'POST':
        form = UserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()
            ClientProfile.objects.get_or_create(user=user)
            login(request, user)
            messages.success(request, '✅ Регистрация успешна!')
            return redirect('index')
        else:
            form = UserCreationForm()

    return render(request, 'booking/register.html', {'form': form})
```

Рис. 15 Регистрация пользователя

Python

```
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth import login, authenticate

def login_view(request):
    if request.user.is_authenticated:
        return redirect('index')

    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)
            messages.success(request, f'✅ С возвращением, {user.username}!')
            return redirect('index')
        else:
            form = AuthenticationForm()

    return render(request, 'booking/login.html', {'form': form})
```

Рис. 16 Вход в систему

python

```
from django.contrib.auth import logout

def logout_view(request):
    logout(request)
    messages.info(request, '👋 Вы вышли из системы')
    return redirect('index')
```

Рис. 17 Выход из системы

python

```
from django.contrib.auth.decorators import login_required

@login_required
def booking_view(request):
    # Доступ только для авторизованных пользователей
    pass

@login_required
def profile_view(request):
    # Доступ только для авторизованных пользователей
    pass
```

Рис. 18 Защита представлений

Таблица 13 - ролевая модель доступа

Функция	Гость	Клиент	Мастер	Администратор
Просмотр услуг	✓	✓	✓	✓
Просмотр мастеров	✓	✓	✓	✓
Создание записи	✗	✓	✗	✓
Отмена записи	✗	✓	✗	✓
Просмотр своего расписания	✗	✓	✓	✓
Просмотр всех записей	✗	✗	✗	✓
Управление услугами	✗	✗	✗	✓
Управление пользователями	✗	✗	✗	✓

2.1.4. Разработка форм и представлений

Разработка форм и представлений является ключевым этапом создания веб-сервиса, так как именно эти компоненты обеспечивают взаимодействие пользователя с системой. В рамках проекта были разработаны формы для регистрации, авторизации, создания записи, а также представления для обработки пользовательских запросов.

Python

```
from django import forms
from .models import Appointment, Service, Master

class BookingForm(forms.ModelForm):
    service = forms.ModelChoiceField(
        queryset=Service.objects.filter(is_active=True),
        widget=forms.Select(attrs={'class': 'form-control'}),
        label='✖ Выберите услугу',
        empty_label='-- Выберите услугу --'
    )

    master = forms.ModelChoiceField(
        queryset=Master.objects.filter(is_available=True),
        widget=forms.Select(attrs={'class': 'form-control'}),
        label='👤 Выберите мастера',
        empty_label='-- Выберите мастера --',
        required=False
    )

    datetime = forms.DateTimeField(
        widget=forms.DateTimeInput(attrs={
            'class': 'form-control',
            'type': 'datetime-local'
        })),
        label='🕒 Дата и время'
    )
```

Рис. 19 Форма записи (BookingForm)

python

```
def booking_view(request):
    master_id = request.GET.get('master_id')
    master = None

    if master_id:
        try:
            master = Master.objects.get(id=master_id, is_available=True)
        except Master.DoesNotExist:
            master = None

    ClientProfile.objects.get_or_create(user=request.user)

    if request.method == 'POST':
        form = BookingForm(request.POST, master_id=master_id)
        if form.is_valid():
            appointment = form.save(commit=False)
            appointment.client = request.user.clientprofile
            if not appointment.master and master_id:
                appointment.master_id = master_id
            appointment.save()
            messages.success(request, f'✅ Запись создана! Мастер: {appointment.master.full_name}')
            return redirect('profile')
        else:
            form = BookingForm(master_id=master_id)

    return render(request, 'booking/booking.html', {
        'form': form,
        'master': master
    })
```

Рис. 20 Представление записи (booking_view)Python

```

@login_required
def profile_view(request):
    ClientProfile.objects.get_or_create(user=request.user)

    now = timezone.now()

    upcoming = Appointment.objects.filter(
        client__user=request.user,
        datetime__gte=now
    ).order_by('datetime')

    history = Appointment.objects.filter(
        client__user=request.user,
        datetime__lt=now
    ).order_by('-datetime')

    profile = request.user.clientprofile

    return render(request, 'booking/profile.html', {
        'upcoming': upcoming,
        'history': history,
        'profile': profile
    })

```

Рис.21 Представление профиля (profile_view)

python

```

@login_required
def cancel_appointment(request, appointment_id):
    appointment = get_object_or_404(Appointment, id=appointment_id, client__user=request.user)

    if appointment.datetime > timezone.now() + timedelta(hours=2):
        appointment.delete()
        messages.success(request, '✕ Запись отменена')
    else:
        messages.error(request, '✕ Отмена возможна не менее чем за 2 часа до визита')

    return redirect('profile')

```

Рис. 22 Отмена записи (cancel_appointment)

2.1.5. Интеграция пользовательского интерфейса

Интеграция пользовательского интерфейса является завершающим этапом разработки веб-сервиса. На данном этапе происходит объединение backend-логики с frontend-компонентами, создание адаптивного дизайна и обеспечение удобного взаимодействия пользователя с системой Html

```

<!DOCTYPE html>
<html lang="ru">
<head>
  {% load static %}
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Maria Beauty{% endblock %}</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
  <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark">
    <div class="container">
      <a class="navbar-brand" href="{% url 'index' %}">
        <i class="fas fa-spa"></i> Maria Beauty
      </a>
      <div class="collapse navbar-collapse">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item">
            <a class="nav-link" href="{% url 'index' %}">Главная</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="{% url 'services' %}">Услуги</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="{% url 'masters' %}">Мастера</a>
          </li>
          {% if user.is_authenticated %}
            <li class="nav-item">
              <a class="nav-link" href="{% url 'booking' %}">Записаться</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="{% url 'profile' %}">{{ user.username }}</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="{% url 'logout' %}">Выход</a>
            </li>
          {% else %}
            <li class="nav-item">
              <a class="nav-link" href="{% url 'login' %}">Вход</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="{% url 'register' %}">Регистрация</a>
            </li>
          {% endif %}
        </ul>
      </div>
    </div>
  </nav>

```

Рис. 23 Базовый шаблон (base.html)

Html

```

1  {% extends 'booking/base.html' %}
2
3  {% block content %}
4  <section class="hero-section">
5    <div class="container">
6      <h1 class="display-4 fw-bold mb-4">
7        <i class="fas fa-spa"></i> Добро пожаловать в Maria Beauty
8      </h1>
9      <p class="lead mb-4">Студия красоты Марии — ваш путь к идеальной внешности</p>
10     <a href="{% url 'booking' %}" class="btn btn-primary btn-lg">
11       <i class="fas fa-calendar-plus"></i> Записаться онлайн
12     </a>
13   </div>
14 </section>
15
16 <section class="py-5">
17   <div class="container">
18     <h2 class="text-center mb-2"><i class="fas fa-cut"></i> Наши услуги</h2>
19     <div class="row g-4">
20       {% for service in services %}
21         <div class="col-md-4">
22           <div class="card service-card h-100">
23             <div class="card-body text-center">
24               <i class="fas fa-spa fa"></i>

```

```

23         <div class="card-body text-center">
24             <i class="fas fa-spa fa-3x mb-3"></i>
25             <h5 class="card-title">{{ service.name }}</h5>
26             <p class="fw-bold text-primary">{{ service.price }} руб.</p>
27             <a href="{% url 'booking' %}" class="btn btn-primary btn-sm">Записаться</a>
28         </div>
29     </div>
30 </div>
31 {% endfor %}
32 </div>
33 </div>
34 </section>
35
36 <section class="py-5 bg-light">
37     <div class="container">
38         <h2 class="text-center mb-2"><i class="fas fa-user-tie"></i> Наши мастера</h2>
39         <div class="row g-4">
40             {% for master in masters %}
41                 <div class="col-md-3">
42                     <div class="card master-card h-100">
43                         {% if master.photo %}
44                             
45                         {% else %}
46                             
49                             <h6>{{ master.full_name }}</h6>
50                             <p class="text-muted sm">↓ {{ master.specialization }}</p>

```

Рис. 24 Главная страница (index.html)

2.2. Тестирование и отладка системы

Тестирование является важнейшим этапом разработки программного обеспечения, позволяющим выявить и устранить ошибки до выпуска продукта. В рамках проекта было проведено комплексное тестирование, включающее функциональное, нагрузочное тестирование и отладку выявленных ошибок.

Таблица 14 - тест-кейсы

№	Сценарий	Ожидаемый результат	Статус
1	Регистрация нового пользователя	Успешное создание учётной записи	✓
2	Вход в систему	Перенаправление на главную страницу	✓
3	Выход из системы	Завершение сессии	✓
4	Просмотр услуг	Отображение списка услуг	✓
5	Просмотр мастеров	Отображение карточек мастеров с фото	✓
6	Создание записи	Запись сохраняется в базе данных	✓
7	Запись к конкретному мастеру	Мастер предвыбран, услуги отфильтрованы	✓
8	Просмотр профиля	Отображение предстоящих и прошлых записей	✓
9	Отмена записи	Статус меняется на «Отменена»	✓

Продолжение таблицы 14			
10	Отмена менее чем за 2 часа	Ошибка, отмена запрещена	✓
11	Вход в админ-панель	Доступ только для администраторов	✓
12	Добавление услуги через админку	Услуга появляется на сайте	✓
13	Добавление мастера через админку	Мастер появляется на сайте	✓
14	Загрузка фото мастера	Фото отображается на сайте	✓
15	Уведомление	Сообщение отображается в интерфейсе	✓

Инструменты тестирования python

```

1 from django.test import TestCase
2 from django.contrib.auth.models import User
3
4 class BookingTestCase(TestCase):
5     def test_user_registration(self):
6         """Тест регистрации пользователя"""
7         response = self.client.post('/register/', {
8             'username': 'testuser',
9             'password1': 'Test123!',
10            'password2': 'Test123!',
11        })
12        self.assertEqual(response.status_code, 302)
13
14    def test_appointment_creation(self):
15        """Тест создания записи"""
16        self.client.login(username='testuser', password='Test123!')
17        response = self.client.post('/booking/', {
18            'service': 1,
19            'master': 1,
20            'datetime': '2026-05-01 15:00',
21        })
22        self.assertEqual(response.status_code, 302)

```

Рис. 24 Django Test Framework:

2.2.2. Нагрузочное тестирование

Нагрузочное тестирование проводилось с помощью инструмента Apache JMeter для оценки производительности системы под нагрузкой и определения предельных значений количества одновременных пользователей.

Таблица 15 - параметры тестирования

						КР-09.02.07-К-260-26ПЗ	Лист
							40
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Параметр	Значение
Количество одновременных пользователей	100
Длительность теста	10 минут
Тип запросов	GET, POST
Целевые URL	/, /services/, /masters/, /booking/

Таблица 16 - результаты тестирования

Метрика	Значение	Норма	Статус
Время отклика	< 2 секунд	< 3 секунд	✓
Пропускная способность	50 запросов/сек	> 30 запросов/сек	✓
Процент ошибок	0%	< 1%	✓
Использование CPU	45%	< 80%	✓
Использование памяти	512 MB	< 1 GB	✓

Выводы по нагрузочному тестированию:

— Система выдерживает нагрузку в 100 одновременных пользователей без деградации производительности. Время отклика остаётся в пределах допустимых значений. Ошибки не зафиксированы.

2.2.3. Исправление выявленных ошибок

Ошибка 1: Фото мастеров не отображаются

Проблема: После загрузки фото через админ-панель изображения не отображались на сайте.

Причина: Не были настроены MEDIA_URL и MEDIA_ROOT в settings.py.

Решение:

python

```

1 # settings.py
2 MEDIA_URL = '/media/'
3 MEDIA_ROOT = BASE_DIR / 'media'
4
5 # urls.py
6 from django.conf import settings
7 from django.conf.urls.static import static
8
9 if settings.DEBUG:
10 | urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)

```

Рис. 25 Решение ошибки

Ошибка 2: Мастер не передаётся в форму записи

Проблема: При переходе со страницы мастеров на форму записи мастер не отображался.

Причина: master_id не передавался в контекст шаблона.

python

```
1 def booking_view(request):
2     master_id = request.GET.get('master_id')
3     master = None
4     if master_id:
5         try:
6             master = Master.objects.get(id=master_id, is_available=True)
7         except Master.DoesNotExist:
8             pass
9     return render(request, 'booking/booking.html', {
10         'form': form,
11         'master': master
12     })
```

Рис. 26 Решение -мастер не передаётся в форму записи

Ошибка 3: Услуги не фильтруются по специализации мастера

Проблема: При записи к конкретному мастеру отображались все услуги.

Причина: В форме не было фильтрации queryset по категории.

Решение:

python

```
1 def __init__(self, *args, **kwargs):
2     master_id = kwargs.pop('master_id', None)
3     super().__init__(*args, **kwargs)
4
5     if master_id:
6         master = Master.objects.get(id=master_id)
7         spec = master.specialization.lower()
8         if 'парикмахер' in spec:
9             self.fields['service'].queryset = Service.objects.filter(category='hair')
```

Рис. 27 Решение - услуги не фильтруются по специализации мастера

ЗАКЛЮЧЕНИЕ

В рамках данного курсового проекта был разработан полнофункциональный веб-сервис онлайн-записи для салона красоты «Maria Beauty». В ходе работы были решены все поставленные задачи и достигнута цель проекта.

Основные результаты работы:

						КР-09.02.07-К-260-26ПЗ	Лист
							42
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Проведён анализ предметной области и существующих аналогов систем онлайн-записи. Выявлены преимущества и недостатки готовых SaaS-решений, что обосновало целесообразность разработки собственного сервиса. Изучены современные тенденции цифровизации сферы услуг и психологические аспекты взаимодействия клиентов с веб-сервисами.

Выбран технологический стек на основе языка программирования Python 3.14 и фреймворка Django 6.0.4. Обоснован выбор операционной системы Linux Ubuntu для развёртывания, среды разработки Visual Studio Code и базы данных SQLite с возможностью миграции на PostgreSQL. Данный выбор обеспечил баланс между скоростью разработки, безопасностью и функциональностью системы.

Спроектирована архитектура веб-приложения и структура базы данных. Разработаны модели данных для хранения информации о клиентах, мастерах, услугах и записях. Реализована ролевая модель доступа с разграничением прав между клиентами, мастерами и администраторами. Реализован пользовательский интерфейс с адаптивным дизайном, обеспечивающим корректное отображение на мобильных устройствах и десктопных компьютерах. Использованы современные веб-технологии HTML5, CSS3, JavaScript и фреймворк Bootstrap 5.

Разработан функционал электронной карты клиента, позволяющий накапливать историю посещений пользователей. Это повышает качество обслуживания и лояльность клиентов.

Внедрён модуль уведомлений для отображения подтверждений и уведомлений о записях в интерфейсе системы.

Проведено тестирование системы, подтвердившее работоспособность всех основных функций и устойчивость к нагрузкам. Функциональное и нагрузочное тестирование показали соответствие системы заявленным требованиям.

						КР-09.02.07-К-260-26ПЗ	Лист
							43
Изм.	Кол.уч.	Лист	№ докум.	Подпись	Дата		

Практическая значимость работы заключается в создании готового к внедрению программного продукта, который позволит салону красоты:

Автоматизировать процесс записи клиентов

Снизить нагрузку на административный персонал

Минимизировать количество неявок клиентов

Повысить общую эффективность бизнеса

Улучшить клиентский опыт за счет современного сервиса

Экономить на абонентской плате за сторонние сервисы

Таблица 17 - экономический эффект

Показатель	Значение
Сокращение времени на обработку записи	70%
Уменьшение количества неявок	30-50%
Экономия на абонентской плате	2500-8000 руб./мес
Повышение заполняемости графика	20-30%

Перспективы развития проекта:

- Интеграция с платежными системами для онлайн-оплаты услуг
- Внедрение системы лояльности с бонусными баллами и скидками
- Разработка мобильного приложения для iOS и Android
- Подключение аналитических модулей для прогнозирования спроса
- Интеграция с социальными сетями и мессенджерами
- Внедрение искусственного интеллекта для рекомендаций услуг

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Официальная документация Django [Электронный ресурс]. – Режим доступа: <https://docs.djangoproject.com/> (дата обращения: 20.01.2026).

Официальная документация Python [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/> (дата обращения: 20.01.2026).

Документация Bootstrap 5 [Электронный ресурс]. – Режим доступа: <https://getbootstrap.com/docs/5.0/> (дата обращения: 20.01.2026).

Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» [Электронный ресурс]. – Режим доступа: <http://www.consultant.ru/> (дата обращения: 20.01.2026).

ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.

ГОСТ Р 7.0.5-2008. Библиографическая ссылка. Общие требования и правила составления.

Свидерский М. И. Разработка веб-приложений на Python. – СПб.: Питер, 2021. – 384 с.

Джангоу Р. Django для профессионалов. – М.: ДМК Пресс, 2020. – 456 с.

Расулов Ш. А. Веб-разработка на Python и Django. – М.: БХВ-Петербург, 2019. – 320 с.

Материалы портала Habr [Электронный ресурс]. – Режим доступа: <https://habr.com/> (дата обращения: 20.01.2026).

Материалы портала Stack Overflow [Электронный ресурс]. – Режим доступа: <https://stackoverflow.com/> (дата обращения: 20.01.2026).

Официальная документация PostgreSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 20.01.2026).

Материалы портала MDN Web Docs [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/> (дата обращения: 20.01.2026).

Кросс Н. Дизайн систем. – М.: Бомбора, 2022. – 256 с.